

Project Proposal

Reproducing and Improving RoLoRA

Daniel Popovici, Vlad Iftode, Sorin Zele

12 May 2026

1. Introduction

Large language models (LLMs) achieve strong performance because they are trained on large and diverse datasets. However, adapting such models to new tasks or domains is expensive: updating all model parameters requires substantial computation, memory, and communication. This becomes especially challenging in settings where training data is distributed across many devices or organizations and cannot be collected in one central location.

Federated learning [1] addresses this problem by allowing multiple clients to train a shared model collaboratively without directly sharing their private data. In a typical federated learning round, a server sends the current global model to N clients. Each client i trains locally on its own data and sends an update back to the server. The server then combines the updates, commonly using FedAVG, to produce the next global model. This approach is attractive for privacy-sensitive applications, but applying it directly to LLMs is costly because full model updates can contain millions or billions of parameters.

Parameter-efficient fine-tuning (PEFT) methods reduce this cost by updating only a small part of the model. Low-Rank Adaptation (LoRA) [2] is one of the most widely used PEFT methods. Instead of modifying a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times d}$ directly, LoRA keeps W_0 frozen and learns a low-rank update,

$$W = W_0 + \alpha \mathbf{A} \mathbf{B},$$

where $\mathbf{A} \in \mathbb{R}^{d \times r}$, $\mathbf{B} \in \mathbb{R}^{r \times d}$, and $r \ll d$. The key idea is that the update to the model can often be represented well using two much smaller matrices. As a result, only $\mathbf{A}$ and $\mathbf{B}$ need to be trained and communicated, which makes LoRA well suited for federated fine-tuning.

However, combining LoRA with federated learning introduces a critical tension between computational and communication efficiency versus mathematical correctness. The true global update requires averaging the clients' full low-rank products: $\frac{1}{N} \sum_{i=1}^N \mathbf{A}_i \mathbf{B}_i$. If clients transmit these full matrices, it destroys LoRA's bandwidth savings. If clients transmit only their small factors ($\mathbf{A}_i$ and $\mathbf{B}_i$), the server still cannot just simply sum them to broadcast the true update. The server must explicitly compute a new rank- r approximation of the aggregated sum, typically requiring Singular Value Decomposition (SVD) or similar projection algorithms which represent computationally intensive operations. Alternatively, the clients could reinitialize their factors every training round, but this introduces initialization lag due to frequent resets and still requires the central server to communicate the full model parameters every round, a huge communication bottleneck [3].

To bypass these bandwidth and compute costs, standard methods simply apply FedAVG to the individual factors separately. However, due to the non-linearity of matrix multiplication, this creates a severe aggregation error:

$$\frac{1}{N} \sum_{i=1}^N \mathbf{A}_i \mathbf{B}_i \neq \left(\frac{1}{N} \sum_{i=1}^N \mathbf{A}_i \right) \left(\frac{1}{N} \sum_{i=1}^N \mathbf{B}_i \right). \quad (1)$$

This means that directly applying FedAVG to LoRA factors can create an aggregated model that does not correctly represent the clients' intended updates. Existing methods try to avoid this issue in different ways. For example, FFA-LoRA [4] freezes one LoRA matrix and trains only the other, which makes aggregation simpler and reduces communication. The drawback

is that freezing one matrix reduces the expressive power of the adapter, especially when the number of trainable parameters is small or the clients’ data distributions differ strongly.

RoLoRA [5] addresses this limitation using alternating optimization. Instead of always freezing the same matrix, RoLoRA alternates between two types of communication rounds. In one round, all clients freeze **A** and train **B**; in the next round, they freeze **B** and train **A**. Because one matrix is shared and fixed during each round, the active matrix can be averaged exactly without introducing the product-of-averages error. At the same time, both LoRA matrices are eventually learned, so RoLoRA preserves more expressiveness than methods that permanently freeze one factor.

The original RoLoRA paper shows that this alternating strategy is both theoretically and empirically effective. Its analysis demonstrates why learning the down-projection matrix is important, and its experiments show strong performance on RoBERTa-Large and Llama-2-7B across several federated fine-tuning tasks [5]. In particular, RoLoRA is more robust than standard FedAVG-LoRA and FFA-LoRA when the number of clients increases, which is one of the most important settings for scalable federated learning.

In this project, we study RoLoRA as a scalable learning system. Our first goal is to reproduce the core experimental findings of the paper, focusing on the reported RoBERTa-Large results in federated GLUE fine-tuning. Our second goal is to investigate whether RoLoRA can be improved using recent ideas from the broader LoRA literature, such as better initialization strategies, alternative optimization choices, or more stable aggregation methods. We aim to evaluate these changes while preserving the central design principle of RoLoRA: alternating between the two LoRA matrices so that communication remains efficient and aggregation remains well defined.

2. Project Plan

This project consists of two main parts. First, we will reproduce the key experimental results reported for RoLoRA. Second, we will investigate a small number of targeted extensions that may improve its performance while preserving the main structure of the method.

Reproduction goal. We will focus on the central RoBERTa-Large experiments from the RoLoRA paper. In particular, we aim to reproduce Table 1, which reports accuracy on five GLUE tasks for different numbers of clients, and Figure 3, which shows convergence curves over communication rounds in the 50-client setting. These results are central to the paper’s main claim: RoLoRA remains robust as the number of clients increases, while FedAVG-LoRA and FFA-LoRA degrade more strongly. We choose RoBERTa-Large because it is the main experimental setting in the paper and is more feasible to reproduce than the larger Llama-2-7B experiments.

Experimental strategy. We will not attempt to reproduce all datasets at once. Instead, we will begin with one GLUE dataset and implement the full comparison between RoLoRA, FedAVG-LoRA, and FFA-LoRA for that dataset. This includes testing different client counts and collecting convergence curves. Once this setup is correct and reproducible, we will extend the experiments to additional datasets where time and compute allow. This incremental approach reduces risk: even a complete reproduction on one dataset provides a useful validation of the method, while additional datasets improve the strength and generality of our results.

Improvement phase. After establishing the reproduction baseline, we will explore possible improvements to RoLoRA. These improvements will be selected based on a focused literature review of recent work on federated LoRA and centralized LoRA variants, as described in Section 3. Candidate ideas will be implemented on top of the reproduced codebase and evaluated against the same baselines. We expect this phase to be iterative: early experimental results will help determine which directions are worth pursuing further.

Implementation and compute. We will use the authors’ released code as the starting point for both reproduction and improvement experiments. Where possible, we will keep the hyperparameters consistent with those reported in the paper to ensure a fair comparison. At this stage, the exact computational cost is still uncertain. We will first test whether the experiments can be run locally; if this is not feasible, we will use the DelftBlue high-performance computing cluster [6].

3. Research Directions

After reproducing the main RoLoRA results, we will investigate whether the method can be improved without changing its central alternating optimization structure. The goal is not to design a completely new federated fine-tuning method, but to study small and well-motivated modifications that are simple to implement and meaningful to evaluate.

RoLoRA alternates between updating the two LoRA matrices. In one communication round, clients freeze the down-projection matrix and update the up-projection matrix. In the next round, they freeze the up-projection matrix and update the down-projection matrix. This design avoids the aggregation error that appears when both LoRA matrices are averaged independently, while still allowing both matrices to be learned over time [5]. Therefore, any proposed improvement should preserve this basic property: only one LoRA matrix should be trained and communicated in each round.

Here we present a few preliminary research directions for potential improvements. **Please note that these directions are exploratory, and our approach may evolve as the research progresses.**

1. **Improved initialization.** Our first direction is to study improved initialization for the LoRA matrices. The RoLoRA paper shows that freezing a poorly initialized down-projection can limit the final model quality. This suggests that the starting point of the LoRA factors may have a large influence on training, especially in low-rank and many-client settings. We will therefore consider replacing the standard random initialization with more structured alternatives, such as orthogonal or SVD-based initialization. The hypothesis is that a better initial down-projection can make the first rounds of training more stable and may reduce the performance gap caused by poor initial representations.
2. **Separate learning rates for the two LoRA matrices.** Building on the insight that the two LoRA matrices serve distinct functional roles [7], our second direction examines the use of decoupled learning rates. Because RoLoRA isolates the updates of these matrices into separate rounds, it naturally allows for phase-specific learning rates. We aim to determine if tuning these rates independently improves either convergence speed or overall accuracy.
3. **Adaptive server-side optimization.** Finally, we may evaluate a lightweight server-side optimizer instead of plain averaging. The original RoLoRA framework aggregates the active LoRA matrix after each communication round. Replacing this aggregation step with an adaptive federated optimizer could improve convergence, especially when client data is heterogeneous. This direction is attractive because it changes only the server update rule and does not require changing the local training procedure or the alternating structure of RoLoRA.

Among these directions, we will prioritize changes that are easy to integrate into the existing implementation and that preserve RoLoRA’s communication advantage. We will compare each proposed modification against the reproduced RoLoRA baseline using the same datasets, client counts, ranks, and evaluation metrics whenever possible.

4. Detailed Planning

The project runs over six weeks, with a first draft deadline at the end of Week 8 and a buffer week (W9) reserved for finishing experiments and implementing feedback before the final report deadline. The workstreams are organized as follows: setup occupies the first week; literature review and reproduction setup run in parallel from the start; improvement work begins as the literature review concludes at the midpoint of Week 5; and report writing begins in Week 7.

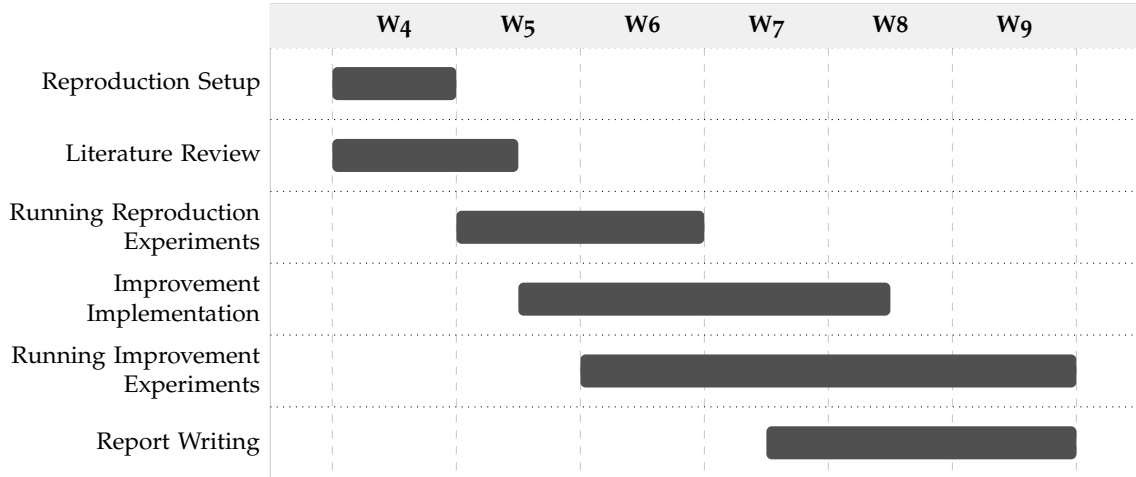


Figure 1: Project timeline. Half week increments.

Improvement implementation and experiments are expected to interleave throughout W5–W8 as results from one direction inform the next. W6 serves as a buffer for any remaining experiments, with report writing running in parallel from W5 onward. A visual diagram of the timeline can be seen in figure 1.

References

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 54 of *Proceedings of Machine Learning Research*, pp. 1273–1282, PMLR, 2017.
- [2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021.
- [3] Z. Bian, Y. Sun, *et al.*, “Lora-fair: Mitigating server-side aggregation bias in federated low-rank adaptation,” 2024.
- [4] Y. Sun, Z. Li, Y. Li, and B. Ding, “Improving LoRA in privacy-preserving federated learning,” in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [5] S. Chen, Y. Guo, Y. Ju, H. Dalal, Z. Zhu, and A. Khisti, “Robust federated finetuning of LLMs via alternating optimization of LoRA,” in *39th Conference on Neural Information Processing Systems (NeurIPS)*, 2025.
- [6] Delft High Performance Computing Centre (DHPC), “DelftBlue Supercomputer (Phase 2).” <https://www.tudelft.nl/dhpc/ark:/44463/DelftBluePhase2>, 2024.
- [7] S. Hayou, N. Ghosh, and B. Yu, “LoRA+: Efficient low rank adaptation of large models,” in *Proceedings of the 41st International Conference on Machine Learning (R. Salakhutdinov,*

Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, eds.), vol. 235 of *Proceedings of Machine Learning Research*, pp. 17783–17806, PMLR, 21–27 Jul 2024.